ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

Môn: CS112 – Phân tích và thiết kế thuật toán

ĐỀ TÀI

DEMO ĐIỀU PHỐI CẢNG CHO HÃNG TÀU

Thuê bến (tô màu đồ thị) · Phân công đội · Quỹ giờ công (cái túi) · Xếp container (đóng thùng)

Giảng viên: **Huỳnh Thị Thanh Thương**
Sinh viên thực hiện: **Võ Thị Thanh Tuyền**
MSSV: **25210357**
Lớp: **CS112.F31.LT.TTNT**
Mã nguồn: https://github.com/SerenaVo-code/portopt

Tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN 4
1.1 Bối cảnh 4
1.2 Lý do chọn bài toán 4
1.3 Ứng dụng thực tế 5
CHƯƠNG 2. MÔ TẢ BÀI TOÁN 6
2.1 Bài toán 1: Thuê bến (tô màu đồ thị khoảng) 6
2.2 Bài toán 2: Phân công đội 6
2.3 Bài toán 3: Quỹ giờ công (cái túi 0/1) 6
2.4 Bài toán 4: Xếp container (đóng thùng) 6
2.5 Liên hệ giữa các module 6
CHƯƠNG 3. MÔ HÌNH BÀI TOÁN 7
3.1 Module 1: Thuê bến 7
3.2 Module 2: Phân công đội 7
3.3 Module 3: Quỹ giờ công (cái túi 0/1) 8
3.4 Module 4: Xếp container (đóng thùng) 8
CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN 10
4.1 Cấu trúc dữ liệu chung và cách sinh bước 10
4.2 Module 1: Thuê bến 10
4.2.1 Cấu trúc dữ liệu 10
4.2.2 Phân tích đặc trưng và ý tưởng giải 10
4.2.3 Thuật toán 11
4.2.4 Độ phức tạp 12
4.3 Module 2: Phân công đội 12
4.3.1 Cấu trúc dữ liệu 12
4.3.2 Phân tích đặc trưng và ý tưởng giải 12
4.3.3 Phát hiện vô nghiệm bằng định lý Hall 13
4.3.4 Độ phức tạp 13
4.4 Module 3: Quỹ giờ công (cái túi 0/1) 13
4.4.1 Cấu trúc dữ liệu 13
4.4.2 Phân tích đặc trưng và ý tưởng giải 14
4.4.3 Độ phức tạp 14
4.5 Module 4: Xếp container (đóng thùng) 14
4.5.1 Cấu trúc dữ liệu 14
4.5.2 Phân tích đặc trưng và ý tưởng giải 15
4.5.3 Độ phức tạp 15
4.6 Tính đúng đắn và cách đánh giá chất lượng 15

CHƯƠNG 5. VÍ DỤ MINH HOẠ CÁC BƯỚC THỰC HIỆN THUẬT TOÁN 17
5.1 Dữ liệu và đồ thị xung đột 17
5.2 Tham lam theo giờ đến 17
5.3 Tham lam chọn hợp đồng rẻ nhất 18
5.4 DSATUR 18
5.5 Quay lui có cắt nhánh 18
5.6 Kết quả tổng hợp 19
CHƯƠNG 6. TƯỜNG THUẬT QUÁ TRÌNH THIẾT KẾ VÀ XÂY DỰNG 20
6.1 Công cụ và phạm vi sử dụng AI 20
6.2 Dòng thời gian 20
6.3 Các prompt đã sử dụng 21
6.4 Vấn đề gặp phải và cách giải quyết 21
6.5 Kiểm chứng, chỉnh sửa và phát triển tiếp kết quả 22
6.6 Quy trình sử dụng AI và kiểm chứng 22
CHƯƠNG 7. THIẾT KẾ THỰC NGHIỆM 24
7.1 Mục tiêu 24
7.2 Cách tạo testcase 24
7.3 Phân nhóm và kích thước 24
7.4 Cách đảm bảo bao phủ các trường hợp 25
7.5 Cách chạy 25
CHƯƠNG 8. ĐÁNH GIÁ 26
8.1 Tính đúng đắn: kết quả chạy testcase 26
8.2 Chất lượng của các thuật toán 26
8.3 Thời gian thực hiện 27
8.4 Giao diện và khả năng minh hoạ từng bước 27
8.5 Hạn chế 27
8.6 Hướng phát triển 28
8.7 Tổng hợp mối liên hệ và kết luận 28
PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ CẤU TRÚC MÃ NGUỒN 30
A.1 Chạy trên Codespaces hoặc trên máy 30
A.2 Cấu trúc thư mục 30
PHỤ LỤC B. PROMPT TIÊU BIỂU TRONG QUÁ TRÌNH PHÁT TRIỂN 31

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1 Bối cảnh

Mỗi ngày, một hãng tàu container phải đưa ra nhiều quyết định cho các chuyến tàu cập cảng: chọn bến và thời gian thuê, phân công đội bốc xếp, sử dụng quỹ giờ tăng ca, rồi bố trí các lô container lên tàu. Đây đều là những bài toán tối ưu tổ hợp có thể mô hình hóa bằng các bài toán kinh điển trong môn Phân tích và thiết kế thuật toán.

Đồ án gồm một giao diện demo cho bốn bài toán. Người dùng có thể chọn thuật toán, nhập dữ liệu, xem từng bước xử lý và so sánh kết quả. Demo sử dụng các hướng tiếp cận như tham lam, quy hoạch động và quay lui có cắt nhánh để minh họa sự khác nhau về chất lượng lời giải và thời gian chạy.

Module	Bài toán thực tế	Mô hình kinh điển	Thuật toán trong demo
1. Thuê bến	Thuê ít tiền nhất cho n tàu, mỗi tàu cần một bến trong khoảng [đến, rời]	Tô màu đồ thị khoảng	Tham lam theo giờ đến, tham lam chọn hợp đồng rẻ nhất, DSATUR, quay lui
2. Phân công đội	Gán đội đủ kỹ năng cho từng lượt tàu, không trùng giờ	Gán việc có ràng buộc (tô màu có danh sách cho phép)	Tham lam theo giờ đến, tham lam tàu khó trước, quay lui + cận
3. Quỹ giờ công	Chọn các yêu cầu tăng ca trong quỹ giờ có hạn	Cái túi 0/1	Tham lam theo giá trị/giờ, quy hoạch động, quay lui + cận
4. Xếp container	Xếp các lô lên ít tàu nhất	Đóng thùng (bin packing)	First-Fit, First-Fit giảm dần, Best-Fit giảm dần, quay lui + cận

Bảng 1.1. Bốn module của demo

1.2 Lý do chọn bài toán

Em chọn đề tài từ ví dụ bài toán Sở thú, một bài toán tô màu đồ thị đã được cô trình bày trên lớp. Em muốn thử áp dụng cách mô hình hóa này vào một bối cảnh có chi phí cụ thể, nên chuyển bài toán sang tình huống hãng tàu thuê bến: mỗi con tàu tương ứng với một đỉnh, mỗi hợp đồng thuê bến tương ứng với một màu, còn các tàu không được dùng chung hợp đồng tạo thành một cạnh xung đột. Mục tiêu là xếp tàu vào các hợp đồng sao cho tổng chi phí thấp nhất.

Em bổ sung mô hình chi phí gồm phí cố định F cho mỗi hợp đồng và đơn giá r tính trên toàn bộ khoảng thời gian hợp đồng bao phủ, kể cả những giờ không có tàu. Vì vậy, phương án dùng ít hợp đồng nhất chưa chắc có tổng chi phí thấp nhất. Sự khác biệt này tạo cơ sở để so sánh kết quả của các thuật toán.

Từ bài toán thuê bến, em mở rộng đề tài sang các quyết định về nhân lực và hàng hóa tại cảng. Demo hiện có bốn module: thuê bến, phân công đội, phân bổ quỹ giờ tăng ca (bài toán cái túi) và xếp container (bài toán đóng thùng). Các module này gắn với những dạng bài và chiến lược đã học trong môn.

Ý tưởng ban đầu xuất phát từ bài giảng CS112 của cô Huỳnh Thị Thanh Thương. Bối cảnh cảng và cách kết nối bốn bài toán thành một demo là phần em tự thiết kế.

1.3 Ứng dụng thực tế

- Trong thực tế, việc xếp lịch tàu và thuê bến cần tránh để các tàu trùng giờ dùng chung một bến. Gom lịch phù hợp vào các hợp đồng có thể giảm số khoản phí cố định. Bài toán có liên hệ với xếp lịch khoảng (interval partitioning) và tô màu đồ thị khoảng.
- Phân công đội bốc xếp, đội xử lý hàng lạnh hoặc đội kiểm định cần xét kỹ năng và đơn giá của từng đội, đồng thời bảo đảm một đội không phục vụ hai tàu cùng lúc.
- Khi quỹ giờ tăng ca có giới hạn, cần chọn các yêu cầu dịch vụ bổ sung sao cho tổng giá trị nhận được lớn nhất mà không vượt quá số giờ cho phép. Đây là bài toán cái túi.
- Khi xếp container, mỗi lô hàng phải được đưa lên trọn vẹn một tàu và tổng khối lượng trên mỗi tàu không vượt quá tải trọng. Mục tiêu là dùng ít tàu nhất; đây là bài toán đóng thùng.

Các bài toán tối ưu tổ hợp thường đòi hỏi phải cân nhắc nhiều phương án. Vì vậy, việc so sánh thuật toán tham lam với quy hoạch động hoặc quay lui có cắt nhánh giúp làm rõ sự đánh đổi giữa thời gian thực hiện và chất lượng lời giải. Thuật toán quay lui chỉ xác nhận được tính tối ưu khi đã duyệt hết các khả năng cần thiết hoặc chứng minh được rằng không nhánh nào có thể cải thiện phương án hiện tại.

CHƯƠNG 2. MÔ TẢ BÀI TOÁN

Chương này lần lượt mô tả dữ liệu, ràng buộc và mục tiêu của từng bài toán.

2.1 Bài toán 1: Thuê bến (tô màu đồ thị khoảng)

Một hãng tàu có n chuyến tàu. Tàu i cần một bến từ giờ s_i đến giờ e_i (s_i < e_i, giờ thập phân, có thể vượt 24 để biểu diễn tàu qua đêm). Hãng thuê bến theo **hợp đồng**. Mỗi hợp đồng chỉ phục vụ các tàu đôi một không chồng giờ. Chi phí một hợp đồng là F cộng r nhân với số giờ tính từ giờ đến của tàu đầu tiên đến giờ rời của tàu cuối cùng, **kể cả giờ trống ở giữa**. Ngoài ra có thể có một số cặp tàu bị ràng buộc thêm, không được ở chung hợp đồng dù không trùng giờ.

Hãy xếp mọi tàu vào các hợp đồng sao cho tổng chi phí nhỏ nhất.

2.2 Bài toán 2: Phân công đội

Có n lượt tàu và m đội. Tàu i cần một tập kỹ năng. Đội j có một tập kỹ năng và đơn giá rate_j mỗi giờ. Gán tàu i cho đội j được phép khi đội j có đủ kỹ năng mà tàu i cần; chi phí là rate_j nhân với thời gian tàu i ở cảng. Một đội không được phục vụ hai tàu trùng giờ, hoặc hai tàu thuộc một cặp bị ràng buộc thêm. Hãy gán mọi tàu cho một đội sao cho tổng chi phí nhỏ nhất, hoặc kết luận không có cách gán hợp lệ.

2.3 Bài toán 3: Quỹ giờ công (cái túi 0/1)

Có n yêu cầu dịch vụ thêm. Yêu cầu i tốn w_i giờ công (số nguyên) và mang lại giá trị v_i. Quỹ giờ công tăng ca là W. Mỗi yêu cầu chỉ được nhận hoặc không nhận (không nhận một phần). Hãy chọn tập yêu cầu có tổng giờ không vượt quá W và tổng giá trị lớn nhất.

2.4 Bài toán 4: Xếp container (đóng thùng)

Có n lô container, lô i nặng w_i tấn. Mỗi tàu chở tối đa C tấn. Mỗi lô phải nằm trọn trên một tàu. Hãy xếp mọi lô lên số tàu ít nhất.

2.5 Liên hệ giữa các module

Cả bốn module dùng chung khung giao diện và cấu trúc dữ liệu cho các bước chạy. Vì vậy, cách theo dõi thuật toán được thiết kế nhất quán giữa các module.

CHƯƠNG 3. MÔ HÌNH BÀI TOÁN

Chương này xác định dữ liệu vào, kết quả cần tìm, các ràng buộc và ví dụ cho từng bài toán. Các phương pháp giải được trình bày ở Chương 4.

3.1 Module 1: Thuê bến

Thành phần	Đặc tả
Input	Danh sách n tàu; mỗi tàu có (s_i, e_i) với $s_i < e_i$. Tham số $F \geq 0$ (phí cố định mỗi hợp đồng), $r \geq 0$ (đơn giá mỗi giờ). Tập P các cặp tàu (a, b) bị cấm chung hợp đồng (có thể rỗng).
Output	Một cách tô màu c: $\{1..n\} \to \{1..K\}$, c(i) là hợp đồng của tàu i. Tổng chi phí cost(c).
Ràng buộc	Nếu hai tàu i, j chồng giờ ($s_i < e_j$ và $s_j < e_i$) hoặc $(i, j) \in P$ thì $c(i) \neq c(j)$.
Mục tiêu	Cực tiểu $\text{cost}(c) = \Sigma_k [F + r \times (\max e_i - \min s_i)]$, lấy trên các tàu i có $c(i) = k$.
Cận dưới	$\omega \cdot F + r \cdot \Sigma(e_i - s_i)$, trong đó ω là số tàu chồng nhau nhiều nhất tại một thời điểm.
Giới hạn demo	$n \leq 200$.

Bảng 3.1. Đặc tả module 1

Nhận xét về cận dưới. Trên đồ thị khoảng, số màu tối thiểu bằng ω, và trong mỗi hợp đồng các tàu không chồng nhau nên khoảng phủ không ngắn hơn tổng thời gian các tàu. Vì vậy chi phí không thể thấp hơn $\omega \cdot F + r \cdot \Sigma(e_i - s_i)$. Cận này không phải lúc nào cũng đạt được (khi có giờ trống bắt buộc hoặc ràng buộc thêm), nên nó chỉ dùng để cắt nhánh và đánh giá mức chênh lệch.

Ví dụ 3.1. (testcase m1-T05) 8 tàu, F = 500, r = 40, cặp cấm (Tàu 8, Tàu 7) và (Tàu 7, Tàu 6).

Tàu	Đến	Rời	Tàu	Đến	Rời
Tàu 1	11:00	19:00	Tàu 5	00:30	05:00
Tàu 2	05:30	11:00	Tàu 6	00:30	04:00
Tàu 3	01:00	03:30	Tàu 7	10:00	14:30
Tàu 4	10:30	12:30	Tàu 8	03:00	08:00

Kết quả tối ưu: 4 hợp đồng, tổng chi phí 3.780 (HĐ 1: Tàu 6, 2, 1; HĐ 2: Tàu 5, 7; HĐ 3: Tàu 3; HĐ 4: Tàu 8, 4). Cận dưới: $\omega = 4$, $\Sigma(e - s) = 35{,}5$ giờ, nên cận $= 4 \times 500 + 40 \times 35{,}5 = 3.420$.

3.2 Module 2: Phân công đội

Thành phần	Đặc tả
Input	n tàu; mỗi tàu có (s_i, e_i) và tập kỹ năng cần need_i. m đội; mỗi đội có tập kỹ năng skills_j và đơn giá rate_j. Tập các cặp tàu bị cấm chung đội (có thể rỗng).
Output	Hàm gán a: tàu → đội, kèm tổng chi phí; hoặc kết luận vô nghiệm.

Thành phần	Đặc tả
Ràng buộc	need_i ⊆ skills_a(i). Hai tàu chồng giờ hoặc thuộc cặp cấm thì không cùng đội.
Mục tiêu	Cực tiểu Σ_i rate_a(i) × (e_i − s_i).
Cận dưới	Σ_i (chi phí của đội rẻ nhất đủ kỹ năng cho tàu i).
Giới hạn demo	n ≤ 60 tàu, m ≤ 12 đội.

Bảng 3.2. Đặc tả module 2

Ví dụ 3.2. (testcase m2-T01) 6 tàu và 4 đội: Đội 1 (buộc dây, 24/giờ), Đội 2 (hóa chất, buộc dây, hàng lạnh, cẩu; 19/giờ), Đội 3 (cẩu, 32/giờ), Đội 4 (cẩu, hóa chất, hàng lạnh; 20/giờ). Tàu 1, 3 và 5 cần buộc dây; Tàu 4 cần hàng lạnh; Tàu 6 cần hóa chất; Tàu 2 không cần kỹ năng; cặp cấm (Tàu 1, Tàu 3). Kết quả tối ưu có tổng chi phí 451,5.

Ví dụ vô nghiệm. (testcase m2-D01) ba tàu cùng neo từ giờ 0 đến giờ 5 và đều cần kỹ năng cẩu, nhưng chỉ có hai đội đủ kỹ năng: Kết quả là "vô nghiệm" (vi phạm điều kiện Hall, mục 4.3.3).

3.3 Module 3: Quỹ giờ công (cái túi 0/1)

Thành phần	Đặc tả
Input	n yêu cầu; mỗi yêu cầu có w_i (số nguyên ≥ 1) và v_i ≥ 0. Quỹ W (số nguyên).
Output	Tập chỉ số S ⊆ {1..n}, tổng giá trị Σ_{i∈S} v_i.
Ràng buộc	Σ_{i∈S} w_i ≤ W.
Mục tiêu	Cực đại Σ_{i∈S} v_i.
Giới hạn demo	n ≤ 25, W ≤ 200, n × (W + 1) ≤ 4000 (kích thước bảng quy hoạch động).

Bảng 3.3. Đặc tả module 3

Ví dụ 3.3. (testcase m3-T04) W = 50; A (10 giờ, 60), B (20 giờ, 100), C (30 giờ, 120). Kết quả tối ưu: chọn {B, C}, tổng giá trị 220. Tham lam theo giá trị/giờ chọn {A, B}, chỉ được 160.

3.4 Module 4: Xếp container (đóng thùng)

Thành phần	Đặc tả
Input	n lô với khối lượng w_i > 0; tải trọng mỗi tàu C > 0, w_i ≤ C.
Output	Phân hoạch các lô thành các nhóm (tàu), kèm số tàu dùng.
Ràng buộc	Tổng khối lượng mỗi nhóm không vượt C.
Mục tiêu	Cực tiểu số nhóm.
Cận dưới	⌈Σ w_i / C⌉.
Giới hạn demo	n ≤ 60.

Bảng 3.4. Đặc tả module 4

Ví dụ 3.4. (testcase m4-T04) C = 10, các lô 2, 5, 6, 2, 5, 1, 6, 7, 2 (tổng 36, cận dưới 4). Kết quả tối ưu 4 tàu. First-Fit theo thứ tự cho sẵn dùng 5 tàu.

CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

Chương này trình bày mô hình dữ liệu và cách cài đặt các thuật toán trong project.

4.1 Cấu trúc dữ liệu chung và cách sinh bước

Mỗi thuật toán trả về một danh sách bước. Mỗi bước gồm loại bước (`kind`: `info`, `assign`, `back`, `prune`, `found`, `done`; module 2 có thêm `dead`, module 3 có `skip`, `cell`, `trace`), thông điệp tiếng Việt (`msg`), trạng thái hiện tại và bảng ứng viên kèm lý do bị loại. Backend FastAPI xử lý các thuật toán của một module trong một lần gọi và trả về toàn bộ danh sách bước. Giao diện React đọc danh sách này từ bộ nhớ nên không cần gọi lại máy chủ khi người dùng chuyển bước.

Giao diện tô sáng dòng mã giả tương ứng, kèm thông điệp và bảng ứng viên. Khi chạy quay lui, giao diện còn hiển thị phương án tốt nhất hiện có và cận của trạng thái đang xét. Mỗi thuật toán trả tối đa 3.000 bước; với dữ liệu lớn, giới hạn này được giảm xuống. Quay lui bị giới hạn ở 400.000 nút và 1,5 giây. Nếu dừng trước khi chứng minh được tối ưu, kết quả mang nhãn “CHƯA CHẮC TỐI ƯU”.

4.2 Module 1: Thuê bến

4.2.1 Cấu trúc dữ liệu

- **Danh sách tàu** `ships[i] = (tên, s, e)`.
- **Đồ thị xung đột** G = (V, E) dưới dạng danh sách kề (tập các đỉnh kề): cạnh (i, j) khi hai tàu chồng giờ hoặc thuộc cặp cấm. Dựng đồ thị tốn $O(n^2)$.
- **Mảng màu** `col[i]` (−1 nếu chưa tô) và với mỗi hợp đồng k: giờ đến đầu `lo[k]`, giờ rời cuối `hi[k]`, số tàu `cnt[k]` (dùng cho tham lam chọn hợp đồng rẻ nhất và quay lui). Chi phí hiện tại được cập nhật dần và hoàn tác được khi quay lui.
- **Thứ tự duyệt** `order`: các tàu sắp theo (giờ đến, giờ rời).

4.2.2 Phân tích đặc trưng và ý tưởng giải

Điều kiện cơ bản của bài toán là các tàu chồng giờ không thể dùng chung hợp đồng. Vì vậy, có thể mô hình hóa mỗi tàu thành một đỉnh, mỗi xung đột thành một cạnh và mỗi hợp đồng thành một màu. Với đồ thị khoảng, cách duyệt theo giờ đến và chọn hợp đồng có số hiệu nhỏ nhất còn phù hợp giúp đạt số hợp đồng tối thiểu. Tuy nhiên, mục tiêu ở đây là tối thiểu hóa chi phí F + r × khoảng thời gian được tính cho mỗi hợp đồng. Một hợp đồng có thể phải tính cả những giờ trống giữa hai tàu, nên phương án dùng ít hợp đồng hơn chưa chắc rẻ hơn. Từ đặc điểm đó, demo so sánh bốn cách giải:

- Tham lam theo giờ đến: chọn hợp đồng có số hiệu nhỏ nhất còn phù hợp. Cách này chạy nhanh và tối thiểu số hợp đồng trên đồ thị khoảng khi không có ràng buộc phụ, nhưng không trực tiếp tối thiểu hóa chi phí.

- Tham lam chọn hợp đồng rẻ nhất: với mỗi tàu, so sánh chi phí tăng thêm nếu xếp vào từng hợp đồng hiện có với chi phí mở hợp đồng mới. Phần tăng thêm khi nối vào hợp đồng đã mở được tính trên toàn bộ thời gian kéo dài, kể cả giờ trống.
- DSATUR: ở mỗi lượt, chọn tàu có nhiều màu đã xuất hiện nhất trong các tàu xung đột với nó; nếu hòa, chọn tàu có nhiều xung đột hơn. Thuật toán này tập trung vào số màu, không xét trực tiếp chi phí hợp đồng.
- Quay lui có cắt nhánh: duyệt tàu theo giờ đến và khởi tạo phương án tốt nhất bằng kết quả tham lam tốt nhất. Với mỗi tàu, thuật toán thử các lựa chọn từ rẻ đến đắt, gồm nối vào hợp đồng hiện có hoặc mở hợp đồng mới. Nhánh tìm kiếm bị loại khi cận dưới của nó không thể cải thiện phương án tốt nhất hiện tại.

4.2.3 Thuật toán

Mã giả tham lam theo giờ đến (theo khung GREEDY trong bài giảng):

```
GreedyColoring(ships, G):
  sắp ships theo giờ đến tăng dần
  for mỗi tàu v theo thứ tự:
    forb = {col[u] : u kề v, u đã tô}
    k = 1; while k ∈ forb do k = k + 1
    col[v] = k                  // hợp đồng nhỏ nhất còn dùng được
  return col
```

Mã giả DSATUR:

```
DSATUR(G):
  for lần = 1 to n do:
    v = tàu chưa tô có |forb(v)| lớn nhất (hoà: deg(v) lớn nhất, rồi chỉ số nhỏ)
    k = hợp đồng nhỏ nhất không thuộc forb(v)
    col[v] = k
  return col
```

Mã giả quay lui có cắt nhánh (theo khung try(i) của bài giảng):

```
best = chi phí của kết quả tham lam tốt nhất; bestCol = kết quả đó
Try(i, cost):                   // i: tàu thứ i theo giờ đến
   if i = n then:               // đã xếp hết tàu
      if cost < best then best = cost, bestCol = col
      return
   for mỗi lựa chọn k của tàu i, từ rẻ đến đắt:
      // k: nối vào hợp đồng đang có (không bị chặn) hoặc mở hợp đồng mới
      d = chi phí tăng thêm của lựa chọn k
      gán tàu i vào k;  used' = số hợp đồng đang mở
      bound = cost + d + F × max(0, ω − used')
      if bound < best then Try(i + 1, cost + d)   // ngược lại: bỏ nhánh
      bỏ gán tàu i                          // lùi lại
```

Cận cắt nhánh gồm chi phí đã phát sinh và khoản phí cố định tối thiểu còn phải trả. Chi phí không giảm khi thêm tàu, còn tổng số hợp đồng cuối cùng ít nhất bằng ω, số tàu chồng giờ lớn nhất. Nếu hiện đã mở used' hợp đồng, khoản phí cố định còn lại ít nhất là $F \times \max(0, \omega - \text{used}')$. Cận này không vượt quá chi phí của lời giải hoàn chỉnh nên không loại nhầm nghiệm tối ưu.

Cận hiện tại chưa tính phần r × thời gian của các tàu chưa xếp, vì vậy có thể cắt nhánh chưa mạnh. Nếu chi phí đạt cận dưới toàn cục ω·F + r·Σ(e − s), thuật toán có thể dừng vì đã chứng minh được tính tối ưu.

4.2.4 Độ phức tạp

Thuật toán	Thời gian	Nhận xét
Dựng đồ thị	$O(n^2)$	Xét mọi cặp tàu
Tham lam theo giờ đến	$O(n \log n + n \cdot k + E)$	k: số hợp đồng, E: số cạnh xung đột. Không đảm bảo tối ưu về chi phí
Tham lam hợp đồng rẻ nhất	$O(n \log n + n \cdot k + E)$	Cũng không đảm bảo tối ưu
DSATUR	$O(n \cdot (n + E))$	Chọn đỉnh bằng duyệt tuần tự
Quay lui có cắt nhánh	$O(k^n)$ tệ nhất	Số nút không vượt số cách phân hoạch n tàu (số Bell); cắt nhánh làm giảm đáng kể trong thực tế

Bảng 4.1. Độ phức tạp module 1

Trong demo, mỗi bước còn phải tạo bảng ứng viên và tính thông tin hiển thị trên giao diện, nên chi phí cài đặt thực tế có thể cao hơn các công thức độ phức tạp trong bảng. Các công thức mô tả thuật toán, không phải số đo thời gian chạy.

Các phần em tự thiết kế gồm mô hình chi phí hợp đồng; cận dưới ω·F + r·Σ(e − s); phần phí cố định trong cận cắt nhánh; thứ tự thử lựa chọn từ rẻ đến đắt; cách khởi tạo quay lui từ kết quả tham lam; và thuật toán tham lam chọn hợp đồng có chi phí tăng thêm thấp nhất.

4.3 Module 2: Phân công đội

4.3.1 Cấu trúc dữ liệu

- Danh sách đội đủ kỹ năng `elig[i]` cho mỗi tàu và thời gian cập `dur[i] = e_i − s_i`; chi phí giao tàu i cho đội j là `rate_j × dur[i]`.
- Tập xung đột `adj[i]`: các tàu chồng giờ với tàu i hoặc thuộc cặp cấm với tàu i.
- Cận theo tàu `minrate[i]` = chi phí nhỏ nhất trong các đội đủ kỹ năng; `suffix[i]` = tổng `minrate` của các tàu từ vị trí i trở đi theo thứ tự duyệt, dùng làm cận.

4.3.2 Phân tích đặc trưng và ý tưởng giải

Mỗi tàu chỉ có thể được giao cho một đội đáp ứng đủ kỹ năng, và hai tàu xung đột không được dùng chung đội. Nếu bỏ qua yêu cầu kỹ năng, bài toán có cấu trúc tương tự tô màu đồ thị khoảng ở module 1. Khi xét kỹ năng, thứ tự xử lý có thể ảnh hưởng đến kết quả: tàu có ít đội phù hợp nên được xem xét sớm để tránh mất các lựa chọn khả thi. Demo so sánh ba thuật toán:

- **Tham lam theo giờ đến lần lượt chọn đội có đơn giá thấp nhất trong số các đội còn rảnh và đủ kỹ năng.**

- **Tham lam tàu khó trước sắp xếp tàu theo số đội hợp lệ tăng dần, để tàu có ít lựa chọn được xét trước, rồi chọn đội rẻ nhất. Cách sắp xếp này có thể tìm được lời giải trong những trường hợp tham lam theo giờ đến bị kẹt.**
- **Quay lui có cắt nhánh dùng tổng chi phí thấp nhất có thể của các tàu chưa xét làm cận. Thuật toán duyệt tàu theo giờ đến, thử các đội đủ kỹ năng từ rẻ đến đắt, bỏ qua những đội tương đương đang chưa được sử dụng và khởi tạo bằng lời giải tham lam tốt nhất. Một nhánh bị loại khi chi phí hiện tại cộng cận của phần còn lại không thể thấp hơn phương án tốt nhất đã tìm được. Theo ghi chú trong mã, thứ tự theo giờ đến cho kết quả tốt hơn cách xếp tàu khó trước ở các bộ dữ liệu lớn.**

4.3.3 Phát hiện vô nghiệm bằng định lý Hall

Nếu đầu vào vô nghiệm, quay lui có thể phải duyệt nhiều trạng thái mới phát hiện được điều đó. Vì vậy, trước khi tìm kiếm, thuật toán kiểm tra một điều kiện cần: tại thời điểm một tàu bắt đầu neo, các tàu đang cùng có mặt phải ghép được với những đội khác nhau có đủ kỹ năng. Thuật toán đường tăng của Kuhn tìm phép ghép này. Nếu không thể ghép hết, định lý Hall cho phép kết luận bài toán vô nghiệm; giao diện đồng thời nêu các tàu gây ra mâu thuẫn. Phép kiểm tra chưa xét các cặp tàu bị cấm thêm, nên vượt qua bước này chưa bảo đảm bài toán có nghiệm. Khi chưa phát hiện mâu thuẫn, thuật toán tiếp tục tìm kiếm như bình thường.

4.3.4 Độ phức tạp

Thuật toán	Thời gian	Nhận xét
Tham lam (hai kiểu)	O(n log n + n·m·d)	d: số xung đột trung bình. Có thể không gán hết được dù có nghiệm
Kiểm tra Hall	O(n·(n + m)·m) mỗi mốc giờ	Ghép hai phía theo Kuhn; điều kiện cần cho tính khả thi
Quay lui có cắt nhánh	$O(m^n)$ tệ nhất	Mỗi tàu thử tối đa m đội; cắt nhánh bằng suffix

Bảng 4.2. Độ phức tạp module 2

Tham lam và quay lui được xây dựng theo các khung trong bài giảng; bước kiểm tra vô nghiệm dựa trên định lý Hall và thuật toán Kuhn. Mô hình cụ thể, cận dưới Σ minc[i], thứ tự ưu tiên tàu có ít lựa chọn và cách áp dụng ghép hai phía cho các tàu cập cảng cùng lúc là phần em tự thiết kế.

4.4 Module 3: Quỹ giờ công (cái túi 0/1)

4.4.1 Cấu trúc dữ liệu

- Bảng dp[i][w] lưu giá trị lớn nhất có thể đạt được khi xét i yêu cầu đầu tiên với quỹ w giờ. Bảng có kích thước (n + 1) × (W + 1). Backend giới hạn n × (W + 1) ≤ 4.000 ô để kiểm soát bộ nhớ và số bước hiển thị.
- **Mảng take[i] dùng để truy vết các yêu cầu đã chọn và ghi lại nhánh mà thuật toán quay lui đang xét.**

4.4.2 Phân tích đặc trưng và ý tưởng giải

Bài toán có cấu trúc con tối ưu: khi xét yêu cầu thứ i với quỹ w, ta chọn phương án tốt hơn giữa việc bỏ yêu cầu đó và việc nhận yêu cầu đó cùng lời giải cho i − 1 yêu cầu còn lại với quỹ w − w_i. Đây là dạng quy hoạch động được trình bày trong tài liệu. Tham lam theo giá trị trên mỗi giờ phù hợp với bài toán cái túi phân số, nhưng không luôn tối ưu khi mỗi yêu cầu chỉ được chọn trọn vẹn hoặc bỏ qua (Ví dụ 4.1). Quay lui sử dụng cận trên tính bằng cách nới lỏng bài toán thành phiên bản phân số.

Thuật toán	Ý tưởng
Tham lam	Sắp theo v/w giảm dần, nhận yêu cầu nào còn vừa quỹ.
Quy hoạch động	dp[i][w] = max(dp[i−1][w], dp[i−1][w − w_i] + v_i) nếu w ≥ w_i; truy vết để lấy tập chọn.
Quay lui + cận	Duyệt yêu cầu theo giá trị/giờ giảm dần; mỗi yêu cầu: nhận hoặc bỏ; cận trên = giá trị hiện có + nới lỏng phân số phần còn lại; bỏ nhánh khi cận ≤ phương án tốt nhất hiện có (bắt đầu từ kết quả tham lam).

Bảng 4.3. Ba thuật toán của module 3

Ví dụ 4.1 (testcase m3-T04, cũng được nêu ở Ví dụ 3.3): tham lam theo tỷ lệ v/w chọn A (6 đơn vị/giờ), sau đó chọn B (5 đơn vị/giờ), nên không còn đủ quỹ cho C và đạt tổng giá trị 160. Phương án tối ưu chọn B và C, đạt 220. Ở testcase lớn m3-L01 (25 yêu cầu, quỹ 100), kết quả tham lam cũng thấp hơn: 970 so với mức tối ưu 997.

4.4.3 Độ phức tạp

Thuật toán	Thời gian	Bộ nhớ
Tham lam	O(n log n)	O(n)
Quy hoạch động	O(n·W)	O(n·W) (cần bảng để truy vết)
Quay lui + cận	Tệ nhất O(2^n)	O(n) ngăn xếp

Bảng 4.4. Độ phức tạp module 3

Độ phức tạp của quy hoạch động phụ thuộc vào giá trị W, nên thường được gọi là thời gian giả đa thức. Khi W lớn, kích thước bảng và thời gian xử lý tăng đáng kể; vì vậy demo giới hạn W ở mức 200.

Bối cảnh "quỹ giờ công tăng ca" và nội dung giải thích ở từng bước là phần em tự thiết kế.

4.5 Module 4: Xếp container (đóng thùng)

4.5.1 Cấu trúc dữ liệu

- **Mảng bin[i] lưu tàu được gán cho lô i; mảng loads[b] lưu tổng khối lượng đã xếp lên tàu b.**
- Cận dưới ⌈Σ w_i / C⌉.

4.5.2 Phân tích đặc trưng và ý tưởng giải

Số tàu cần dùng ít nhất là ⌈Σ w_i / C⌉. Các thuật toán heuristic dưới đây lần lượt thử xếp từng lô vào một tàu còn chỗ:

- **First-Fit (FF):** xếp lô vào tàu đầu tiên còn đủ chỗ, theo thứ tự cho sẵn.
- **First-Fit Decreasing (FFD):** sắp lô giảm dần theo khối lượng rồi First-Fit.
- **Best-Fit Decreasing (BFD):** sắp giảm dần, xếp vào tàu vừa khít nhất (chỗ còn lại nhỏ nhất).
- **Quay lui có cắt nhánh lấy lời giải tốt nhất trong ba heuristic làm phương án khởi đầu. Thuật toán xét các lô theo khối lượng giảm dần, thử xếp mỗi lô vào những tàu đang mở còn đủ tải rồi mới cân nhắc mở tàu mới. Để tránh xét các trường hợp tương đương, thuật toán chỉ thử một tàu trong số những tàu đang có cùng mức tải. Cận dưới của nhánh bằng số tàu đã mở cộng số tàu tối thiểu cần mở thêm để chứa phần khối lượng còn lại sau khi tận dụng chỗ trống hiện có. Nếu cận này không thể cải thiện lời giải tốt nhất, nhánh bị loại. Khi đạt cận dưới toàn cục ⌈Σ w / C⌉, thuật toán dừng vì đã tìm được phương án tối ưu.**

Sắp xếp lô theo khối lượng giảm dần giúp xử lý các lô khó bố trí trước. Vì vậy FFD và BFD thường cho kết quả tốt hơn FF; ở Ví dụ 3.4, FF cần 5 tàu trong khi FFD chỉ cần 4.

4.5.3 Độ phức tạp

Thuật toán	Thời gian	Chất lượng
FF	$O(n^2)$ (duyệt các tàu có sẵn)	Không quá khoảng 1,7 lần tối ưu
FFD, BFD	$O(n \log n + n^2)$	Không quá khoảng 11/9 lần tối ưu cộng hằng số
Quay lui + cận	Tệ nhất mũ	Tối ưu nếu chạy xong

Bảng 4.5. Độ phức tạp và chất lượng module 4

FF đặt từng lô vào tàu đầu tiên còn đủ tải; FFD sắp lô giảm dần trước khi áp dụng FF; BFD chọn tàu còn ít chỗ nhất nhưng vẫn chứa được lô. Cận dưới cơ bản là tổng khối lượng chia cho tải trọng mỗi tàu, làm tròn lên. Quay lui bỏ qua các tàu có cùng mức tải và phát thông điệp theo từng bước.

4.6 Tính đúng đắn và cách đánh giá chất lượng

- Các thuật toán tham lam không bảo đảm tìm được phương án tối ưu trong cả bốn module. Điều này thể hiện ở Ví dụ 3.1 (module 1: chi phí 3.960 so với mức tối ưu 3.780), testcase m2-D02 (module 2: tham lam theo giờ đến không tìm được cách gán dù bài toán có nghiệm), Ví dụ 4.1 (module 3) và Ví dụ 3.4 (module 4). Những trường hợp này là cơ sở để so sánh các chiến lược giải khác nhau.
- Quay lui có cắt nhánh chỉ mang nhãn "TỐI ƯU" khi đã tìm xong hoặc chứng minh được lời giải tối ưu. Mã nguồn giới hạn mỗi lần chạy ở 400.000 nút và 1,5 giây. Nếu chạm giới hạn trước khi có chứng minh, kết quả được ghi là "CHƯA CHẮC TỐI ƯU".

- **Quy hoạch động** (module 3) cho kết quả tối ưu vì đúng cấu trúc con tối ưu.
- Cận dưới hoặc cận trên được dùng để ước lượng khoảng cách giữa kết quả hiện tại và giá trị tối ưu.

CHƯƠNG 5. VÍ DỤ MINH HOẠ CÁC BƯỚC THỰC HIỆN THUẬT TOÁN

Chương này phân tích testcase m1-T05 của module 1, cũng được giới thiệu ở Ví dụ 3.1. Đây là trường hợp cả bốn thuật toán đều dùng 4 hợp đồng nhưng cho tổng chi phí khác nhau. Các bước và thông điệp được lấy từ dữ liệu demo trả về, nên có thể đối chiếu trực tiếp trên giao diện. Để xem lại ví dụ, nhập testcase m1-T05 vào module 1 (F = 500, r = 40, cặp cấm gồm Tàu 8 với Tàu 7 và Tàu 7 với Tàu 6), sau đó lần lượt chọn từng thuật toán.

5.1 Dữ liệu và đồ thị xung đột

Theo giờ đến, thứ tự xử lý là Tàu 6, 5, 3, 8, 2, 7, 4 rồi Tàu 1. Đồ thị xung đột có 14 cạnh: (1,4), (1,7), (2,4), (2,7), (2,8), (3,5), (3,6), (3,8), (4,7), (5,6), (5,8), (6,7), (6,8) và (7,8). Hai cạnh (6,7) và (7,8) xuất phát từ các cặp bị cấm, dù mỗi cặp tàu không chồng giờ. Bậc của các đỉnh lần lượt là: Tàu 1: 2; Tàu 2: 3; Tàu 3: 3; Tàu 4: 3; Tàu 5: 3; Tàu 6: 4; Tàu 7: 5; Tàu 8: 5. Có tối đa ω = 4 tàu cùng neo trong khoảng từ giờ 3 đến giờ 3,5 (Tàu 3, 5, 6 và 8). Tổng thời gian neo là 35,5 giờ, nên cận dưới bằng 4 × 500 + 40 × 35,5 = 3.420.

5.2 Tham lam theo giờ đến

Mỗi tàu lần lượt được xếp vào hợp đồng có số hiệu nhỏ nhất còn phù hợp; thuật toán không xét chi phí tăng thêm.

Bước	Tàu xét (đến–rời)	Hợp đồng bị chặn	Chọn	Tăng thêm	Tổng
1	Tàu 6 (00:30–04:00)	không có	HĐ 1	640	640
2	Tàu 5 (00:30–05:00)	HĐ 1	HĐ 2	680	1320
3	Tàu 3 (01:00–03:30)	HĐ 1, 2	HĐ 3	600	1920
4	Tàu 8 (03:00–08:00)	HĐ 1, 2, 3	HĐ 4	700	2620
5	Tàu 2 (05:30–11:00)	HĐ 4	HĐ 1	280	2900
6	Tàu 7 (10:00–14:30)	HĐ 1, 4	HĐ 2	380	3280
7	Tàu 4 (10:30–12:30)	HĐ 1, 2	HĐ 3	360	3640
8	Tàu 1 (11:00–19:00)	HĐ 2, 3	HĐ 1	320	3960

Bảng 5.1. Tham lam theo giờ đến: 4 hợp đồng, chi phí 3.960

Tàu 2 được xếp vào Hợp đồng 1 vì đây là hợp đồng có số hiệu nhỏ nhất còn phù hợp. Tuy nhiên, cách xếp này tạo 1,5 giờ trống giữa lúc Tàu 6 rời bến (04:00) và Tàu 2 đến (05:30). Tương tự, việc xếp Tàu 7 vào Hợp đồng 2 tạo 5 giờ trống, còn Tàu 4 vào Hợp đồng 3 tạo 7 giờ trống. Kết quả vẫn dùng đúng ω = 4 hợp đồng, nhưng tổng chi phí là 3.960, cao hơn phương án tối ưu 180.

5.3 Tham lam chọn hợp đồng rẻ nhất

Với mỗi tàu, thuật toán so sánh chi phí tăng thêm khi nối vào một hợp đồng hiện có, kể cả giờ trống, với chi phí mở hợp đồng mới F + r × thời gian neo của tàu.

Bước	Tàu xét	Quyết định	Chi phí	Tổng
1	Tàu 6	Chưa có hợp đồng, mở HĐ 1	500 + 40×3,5 = 640	640
2	Tàu 5	Bị chặn bởi HĐ 1; mở HĐ 2	500 + 40×4,5 = 680	1320
3	Tàu 3	Bị chặn bởi HĐ 1, 2; mở HĐ 3	500 + 40×2,5 = 600	1920
4	Tàu 8	Bị chặn bởi HĐ 1, 2, 3; mở HĐ 4	500 + 40×5 = 700	2620
5	Tàu 2	Nối HĐ 2 (rảnh từ 05:00, trống 0,5 giờ)	40×6 = 240 < mở mới 720	2860
6	Tàu 7	Nối HĐ 3 (rảnh từ 03:30, trống 6,5 giờ)	40×11 = 440 < mở mới 680	3300
7	Tàu 4	Nối HĐ 4 (rảnh từ 08:00, trống 2,5 giờ)	40×4,5 = 180 < mở mới 580	3480
8	Tàu 1	Nối HĐ 2 (rảnh từ 11:00, trống 0 giờ)	40×8 = 320 < mở mới 820	3800

Bảng 5.2. Tham lam hợp đồng rẻ nhất: 4 hợp đồng, chi phí 3.800

5.4 DSATUR

DSATUR chọn tàu có nhiều hợp đồng bị chặn nhất. Nếu có nhiều tàu cùng mức, thuật toán ưu tiên tàu có nhiều xung đột hơn.

Bước	Tàu chọn	Bị chặn / xung đột	Hợp đồng	Tăng thêm
1	Tàu 7	0 hợp đồng / 5 xung đột	HĐ 1	680
2	Tàu 8	1 (HĐ 1) / 5	HĐ 2	700
3	Tàu 6	2 (HĐ 1, 2) / 4	HĐ 3	640
4	Tàu 2	2 (HĐ 1, 2) / 3	HĐ 3	280
5	Tàu 3	2 (HĐ 2, 3) / 3	HĐ 1	360
6	Tàu 5	3 (HĐ 1, 2, 3) / 3	HĐ 4	680
7	Tàu 4	2 (HĐ 1, 3) / 3	HĐ 2	180
8	Tàu 1	2 (HĐ 1, 2) / 2	HĐ 3	320

Bảng 5.3. DSATUR: 4 hợp đồng, chi phí 3.840

DSATUR cũng dùng ω = 4 hợp đồng, nhưng tổng chi phí là 3.840, cao hơn mức tối ưu 3.780. Nguyên nhân là thuật toán ưu tiên số hợp đồng và không xét chi phí của khoảng thời gian trống giữa các tàu trong cùng hợp đồng.

5.5 Quay lui có cắt nhánh

Quay lui khởi đầu với phương án tốt nhất hiện có từ thuật toán tham lam chọn hợp đồng rẻ nhất, có chi phí 3.800. Thuật toán trả về 108 bước, gồm 27 bước thử, 27 bước quay lui, một lần tìm

thấy phương án tốt hơn, 51 lần bỏ nhánh, cùng bước bắt đầu và kết thúc. Tổng cộng có 78 lần thử gán tàu. Bảng sau tóm tắt các giai đoạn chính.

Giai đoạn	Diễn biến	Phương án tốt nhất
1. Đi xuống sâu nhất	Thử lần lượt Tàu 6→HĐ 1 (640), Tàu 5→HĐ 2 (1320), Tàu 3→HĐ 3 (1920), Tàu 8→HĐ 4 (2620), Tàu 2→HĐ 2 (2860), Tàu 7→HĐ 3 (3300), Tàu 4→HĐ 4 (3480). Với Tàu 1, mọi lựa chọn đều cho cận không thấp hơn 3800 (HĐ 2: 3800; HĐ 1: 4080; mở mới: 4300): bỏ nhánh cả ba.	3800 (từ tham lam)
2. Lùi và thử lại ở các tàu cuối	Lùi tới Tàu 4: thử HĐ 1 (3640) nhưng Tàu 1 vẫn bị bỏ nhánh; mở hợp đồng mới cho Tàu 4 cho cận 3880 ≥ 3800: bỏ nhánh. Lùi tới Tàu 7 thử mở hợp đồng mới (3540), rồi các lựa chọn của Tàu 4 và Tàu 1 phía dưới đều bị bỏ nhánh.	3800
3. Nhánh mới tìm được phương án rẻ hơn	Lùi tới Tàu 2, thử HĐ 1 (2900); Tàu 7→HĐ 2 (3280); Tàu 4→HĐ 4 (3460); Tàu 1→HĐ 1 (3780). Đã xếp hết: tìm được phương án 4 hợp đồng, 3780 < 3800.	3780
4. Duyệt nốt	Các nhánh còn lại của Tàu 2, 7, 4 và 1 đều bị bỏ nhánh vì cận không thấp hơn 3780. Tàu 8, 3, 5, 6 không còn lựa chọn khác, nên lùi hết về Tàu 6 và duyệt xong toàn bộ cây tìm kiếm.	3780 (tối ưu)

Bảng 5.4. Các giai đoạn của quay lui

Cận dưới toàn cục là 3.420, thấp hơn chi phí 3.780 nên thuật toán tiếp tục duyệt cây tìm kiếm. Thuật toán hoàn tất sau 78 lần thử, trước khi chạm giới hạn 400.000 nút hoặc 1,5 giây. Vì vậy, kết quả được đánh dấu "TỐI ƯU". Chi phí tìm được cũng trùng với đáp án vét cạn của testcase.

5.6 Kết quả tổng hợp

Thuật toán	Số hợp đồng	Tổng chi phí	So với tối ưu 3780
Tham lam theo giờ đến	4	3960	+180 (+4,8%)
Tham lam hợp đồng rẻ nhất	4	3800	+20 (+0,5%)
DSATUR	4	3840	+60 (+1,6%)
Quay lui có cắt nhánh	4	**3780**	0

Bảng 5.5. Kết quả bốn thuật toán trên ví dụ m1-T05

Hình 5.1. Lời giải tối ưu (3.780): HĐ 1 gồm Tàu 6, 2, 1; HĐ 2 gồm Tàu 5, 7; HĐ 3 gồm Tàu 3; HĐ 4 gồm Tàu 8, 4. Chi phí từng hợp đồng: 1240, 1060, 600, 880.

Tổng bốn hợp đồng: (500 + 40×18,5) + (500 + 40×14) + (500 + 40×2,5) + (500 + 40×9,5) = 1240 + 1060 + 600 + 880 = 3.780.

CHƯƠNG 6. TƯỜNG THUẬT QUÁ TRÌNH THIẾT KẾ VÀ XÂY DỰNG

Chương này ghi lại cách em sử dụng AI trong quá trình phát triển, một số prompt tiêu biểu và cách em kiểm tra kết quả. Số liệu kiểm thử được đối chiếu với mã nguồn hiện tại và nhật ký chạy project.

6.1 Công cụ và phạm vi sử dụng AI

Công cụ AI được ghi nhận trong nhật ký là Claude Code của Anthropic. Bản ghi không lưu tên model hoặc phiên bản model cụ thể. AI hỗ trợ trao đổi yêu cầu, viết và chỉnh sửa mã, xây dựng testcase, chạy kiểm thử và soạn tài liệu. Project dùng FastAPI ở backend và React 18.3.1 cùng Vite 5.4 ở frontend; các thư viện và lệnh cài đặt được liệt kê trong Phụ lục A. Không có framework riêng để điều phối quy trình AI được ghi nhận.

Công đoạn	AI hỗ trợ	Người học thực hiện
Định hướng và mô hình	Giải thích, đưa gợi ý khi được hỏi.	Đặt bối cảnh hãng tàu, mô hình chi phí và phạm vi bốn module; viết prompt và quyết định yêu cầu sản phẩm.
Cài đặt	Hỗ trợ viết và chỉnh mã backend, giao diện, bước diễn giải thuật toán.	Nêu yêu cầu chức năng, xem kết quả và tiếp tục chỉ dẫn khi cần điều chỉnh.
Thiết kế testcase	Sinh testcase, bộ chạy và checker; ban đầu phạm vi có cả kiểm thử API.	Tham gia soạn/chọn tình huống; yêu cầu giới hạn vào hàm thuật toán và không kiểm thử lớp API.
Kiểm tra và sửa lỗi	Chạy pytest, testcase, build và E2E; xử lý các lỗi được phát hiện.	Xem kết quả, hỏi nguyên nhân khi phát hiện e2e.png trong kho mã, và định hướng cách sửa qua prompt.
Tài liệu	Hỗ trợ soạn và cập nhật tài liệu theo trạng thái code.	Đối chiếu yêu cầu môn học, yêu cầu sửa tài liệu và xác nhận thông tin cá nhân/phạm vi sản phẩm.

Bảng 6.1. Phân vai giữa AI và em

6.2 Dòng thời gian

Mốc	Diễn biến	Bằng chứng hiện có
10/10/2026 (theo ghi chép)	Giai đoạn đầu phát triển module thuê bến; lời nhắn nguyên văn không còn trong bản lưu.	Mốc tóm tắt trong tài liệu project.
12–13/10/2026	Mở rộng các module; xây bộ testcase; thu hẹp phạm vi kiểm thử vào hàm thuật toán; xử lý ảnh E2E trong kho mã.	Các prompt tiêu biểu ở Phụ lục B; testcases/RESULTS.md.
15/10/2026	Ghi lại prompt, kết quả và bối cảnh đóng góp trong các tài liệu project.	PROMPTS.md và PROMPT_DECISION_HISTORY.md.

Mốc	Diễn biến	Bằng chứng hiện có
18/10/2026	Trạng thái kiểm thử hiện hành: 88 testcase, 102 pytest, build frontend và E2E đạt.	LOG.md và testcases/RESULTS.md.
Khi lập báo cáo	Đối chiếu nội dung với code và số liệu hiện tại; loại các số liệu/path cũ.	backend/, frontend/, tests/, testcases/ và tài liệu gốc.

Bảng 6.2. Các giai đoạn thực hiện

6.3 Các prompt đã sử dụng

Phụ lục B ghi một số prompt tiêu biểu còn lưu từ phiên làm việc ngày 12–13/10. Các tin nhắn tiếng Việt được chép theo bản ghi khi còn; riêng prompt UI do em xác nhận lại. Phần phản hồi của AI và quyết định của em được tóm tắt theo tài liệu hiện có.

Em tự chuẩn bị thiết kế giao diện và xác nhận đã dùng prompt: “Mình đã thiết kế giao diện trong các hình đính kèm. Hãy xem kỹ bố cục, màu sắc, kiểu chữ, khoảng cách và cách sắp xếp các thành phần, rồi chỉnh giao diện trong project để bám sát thiết kế đó”. Em cung cấp lại câu chữ này để bổ sung vào báo cáo; thời điểm gửi ban đầu không còn trong bản lưu. Ảnh thiết kế nằm trong docs/ui-reference. Báo cáo chưa đối chiếu từng ảnh với giao diện cuối nên không kết luận mức độ tương đồng.

- Ở prompt “viết test case giúp mình”
- Em yêu cầu bộ kiểm thử chỉ gọi trực tiếp các hàm thuật toán, không kiểm thử lớp API. Runner và dữ liệu được điều chỉnh theo phạm vi đó. Bộ hiện có 88 testcase; dữ liệu sai định dạng không thuộc phạm vi kiểm thử này.
- Khi thấy ảnh e2e.png trong kho mã, em hỏi vì sao nó được tạo ra. AI giải thích ảnh do kịch bản E2E chụp và bị thêm vào cùng các thay đổi khác. Sau đó, ảnh được gỡ khỏi kho mã; quy tắc loại trừ được bổ sung và script E2E được chỉnh để lưu ảnh vào thư mục tạm.

Tin nhắn nguyên văn trong giai đoạn đầu xây dựng module 1 không còn trong bản lưu. Báo cáo chỉ nêu các yêu cầu và kết quả có căn cứ từ tài liệu hiện còn.

6.4 Vấn đề gặp phải và cách giải quyết

Vấn đề	Nguyên nhân	Cách xử lý / trạng thái
Testcase đầu tiên có cả kiểm thử API ngoài phạm vi mong muốn.	Yêu cầu ban đầu chưa giới hạn tầng kiểm thử.	Người học yêu cầu chỉ kiểm tra hàm thuật toán; bộ runner được chỉnh để gọi trực tiếp solver.
Một số mô tả testcase không phản ánh kết quả chạy.	Mô tả ban đầu dựa trên dự đoán.	Chạy bộ testcase, tính lại kết quả độc lập và cập nhật mô tả theo dữ liệu trong RESULTS.md.
Ảnh e2e.png bị đưa vào kho mã.	Ảnh do kịch bản E2E tạo và bị thêm cùng thay đổi khác.	Bỏ ảnh khỏi kho, thêm quy tắc loại trừ và chuyển ảnh sinh ra sang thư mục tạm.
Quay lui có thể dừng trước khi chứng minh tối ưu ở dữ liệu lớn.	Code có giới hạn 400.000 nút và deadline 1,5 giây.	Gắn trạng thái chưa chắc tối ưu; RESULTS.md nêu rõ các ca chạm giới hạn.

Vấn đề	Nguyên nhân	Cách xử lý / trạng thái
Phát hiện vô nghiệm ở module 2 chưa bao quát mọi ràng buộc phụ.	Kiểm tra ghép hai phía là điều kiện cần, chưa thay thế toàn bộ tìm kiếm.	Giữ quay lui làm bước tìm kiếm; ghi nhận mở rộng kiểm tra cặp cấm là hướng phát triển.
Mục lục và các kết quả trong báo cáo cũ không còn khớp code.	Bản báo cáo trước còn số liệu và đường dẫn không khớp project hiện tại.	Đối chiếu lại LOG.md, RESULTS.md và cấu trúc mã hiện tại rồi cập nhật báo cáo.

Bảng 6.3. Các vấn đề và cách xử lý

6.5 Kiểm chứng, chỉnh sửa và phát triển tiếp kết quả

Theo LOG.md hiện tại, project đạt 102 bài pytest, 88/88 testcase, frontend build thành công và kịch bản E2E Playwright cho bốn module đạt. Checker của testcase kiểm tra tính hợp lệ, tự tính lại chi phí/giá trị/số thùng, rồi so sánh với đáp án tham chiếu nếu có. Đáp án tham chiếu đến từ vét cạn, công thức, quy hoạch động độc lập hoặc bộ dữ liệu được chuẩn bị sẵn, không lấy từ solver đang được kiểm tra.

Em định hướng đề tài và mô hình chi phí, viết prompt, quyết định phạm vi kiểm thử, tham gia soạn testcase, yêu cầu chỉ kiểm thử các hàm thuật toán và hỏi nguyên nhân khi thấy ảnh kiểm thử trong kho mã. Em tiếp tục yêu cầu chỉnh sửa dựa trên kết quả đã xem.

Việc kiểm chứng được ghi nhận bằng bộ pytest, checker độc lập, testcase, build frontend và kịch bản E2E; báo cáo không khẳng định đã thực hiện thêm một phép kiểm tra thủ công cụ thể nếu không có ghi chép.

Sau module 1, demo được mở rộng thành bốn module. Các thuật toán trả về danh sách bước để phát lại; bộ testcase dùng đáp án độc lập với solver khi có thể xác định được.

6.6 Quy trình sử dụng AI và kiểm chứng

Demo hiện có đặc tả trong spec/, mã backend trong backend/, giao diện trong frontend/, bộ kiểm thử trong tests/ và testcases/. PROMPTS.md lưu các prompt còn ghi nhận; nhật ký không đại diện cho toàn bộ lịch sử trò chuyện.

AI hỗ trợ một phần việc triển khai mã, xây dựng testcase và soạn tài liệu. Em đưa ra yêu cầu, viết prompt, chốt phạm vi testcase và yêu cầu chỉnh sửa khi kết quả chưa phù hợp. Trong báo cáo, em chỉ nhận những quyết định có prompt hoặc xác nhận tương ứng. Nhật ký hiện không lưu tên model cụ thể.

Theo nhật ký hiện có, 88/88 testcase và 102 bài pytest đều đạt; frontend build thành công, kịch bản E2E Playwright cũng chạy qua bốn module. Checker tự tính lại kết quả thay vì dùng số liệu solver trả về. Trong quá trình làm, em yêu cầu bỏ phần kiểm thử API khỏi bộ testcase và hỏi nguyên nhân ảnh e2e.png xuất hiện trong kho mã.

Các lựa chọn thiết kế hiện tại ưu tiên khả năng kiểm chứng bằng testcase độc lập và khả năng quan sát từng bước; kết quả tối ưu chỉ được khẳng định khi có đáp án tham chiếu hoặc thuật toán chứng minh đã hoàn tất.

Hạng mục	Cách triển khai hiện tại	Mục đích
Giới hạn quay lui	Tối đa 400.000 nút và deadline 1,5 giây.	Giới hạn thời gian/tài nguyên; khi dừng sớm, không khẳng định tối ưu.
Sinh bước	Tối đa 3.000 bước, có điều chỉnh theo kích thước input.	Giữ phản hồi và giao diện phát lại ở mức dùng được.
Module 2	Kiểm tra ghép hai phía trước tìm kiếm quay lui.	Phát hiện nhanh một số trường hợp không thể phân công.
Quỹ giờ công	Bảng quy hoạch động bị giới hạn theo $n\times(W+1)\leq 4.000$ ô.	Bảo đảm kích thước dữ liệu phù hợp với giao diện từng bước.
Dữ liệu mẫu	Các mẫu giao diện được tạo từ testcase.	Dùng chung nguồn dữ liệu giữa demo và kiểm thử.
Nhập/xuất	JSON theo cấu trúc input của testcase.	Tái sử dụng dữ liệu để chạy lại và kiểm tra.
Checker	Xác minh ràng buộc và tính lại kết quả độc lập.	Không phụ thuộc số liệu solver tự báo.

Các hạn chế của phiên bản được chọn để nộp được trình bày ở mục 8.5.

CHƯƠNG 7. THIẾT KẾ THỰC NGHIỆM

7.1 Mục tiêu

- Mục tiêu thứ nhất là xác minh các hàm thuật toán trả về lời giải hợp lệ trên nhiều dạng dữ liệu. Bộ testcase gọi trực tiếp các hàm này, không kiểm thử lớp API.
- Mục tiêu thứ hai là so sánh chất lượng lời giải và thời gian chạy giữa các thuật toán trong cùng một module.
- Mục tiêu thứ ba là xây dựng các ví dụ cho thấy thuật toán tham lam có thể không tìm được phương án tối ưu, từ đó minh họa rõ hơn sự khác nhau giữa các phương pháp.

7.2 Cách tạo testcase

Em cùng AI xây dựng bộ testcase và tham gia soạn, chọn tình huống. Sau khi xem bộ đầu tiên, em giới hạn phạm vi kiểm thử ở các hàm thuật toán, không bao gồm API hay dữ liệu sai định dạng. Bộ hiện tại có 88 testcase JSON, chia theo bốn module và năm nhóm. Công cụ tools/gen_testcases.py dùng hạt giống ngẫu nhiên cố định để có thể sinh lại dữ liệu. Các ca viết tay và bộ dữ liệu có cấu trúc giúp kiểm tra những tình huống cụ thể. Mỗi testcase có mã, nhóm, mô tả, dữ liệu đầu vào và kết quả mong đợi. Đáp án tham chiếu được xác định riêng, không lấy từ solver đang kiểm tra.

7.3 Phân nhóm và kích thước

Nhóm	Module 1	Module 2	Module 3	Module 4	Tổng
Thông thường	5	4	4	3	16
Dữ liệu nhỏ	4	3	3	3	13
Biên	9	5	5	5	24
Đặc biệt	6	5	4	4	19
Dữ liệu lớn	5	4	3	4	16
Tổng	29	21	19	19	88

Bảng 7.1. Số testcase theo nhóm và module

Nhóm	Quy mô trong bộ hiện tại	Nội dung được kiểm
Thông thường	3–14 phần tử, tùy module	Dữ liệu phổ biến, mẫu chính và dữ liệu ngẫu nhiên có seed.
Dữ liệu nhỏ	1–5 phần tử	Ca tối thiểu và ca đủ nhỏ để xác định đáp án bằng vét cạn.
Biên	1–20 phần tử	Thời điểm chạm nhau, chồng sát, chi phí/quỹ bằng 0, tàu qua đêm, số thập phân, đúng tải/đúng quỹ.
Đặc biệt	3–10 phần tử	Bẫy tham lam, hòa, phần tử trùng, ràng buộc cặp, vô nghiệm/thiếu kỹ năng hoặc thiếu đội.
Dữ liệu lớn	18–200 phần tử	Ca ngẫu nhiên, ca đạt giới hạn input và ca có cấu trúc/đáp án dựng sẵn.

Bảng 7.2. Mô tả các nhóm testcase

Bộ 88 testcase không bao gồm dữ liệu sai định dạng vì runner gọi trực tiếp hàm thuật toán, không đi qua lớp API/kiểm tra schema. Vì vậy, kết quả này không đánh giá xử lý HTTP 422 hoặc xác thực input của API.

7.4 Cách đảm bảo bao phủ các trường hợp

- **Đáp án tham chiếu được xác định độc lập với solver: dùng vét cạn cho dữ liệu nhỏ; công thức cho các trường hợp đặc biệt như F = 0, r = 0 hoặc các tàu nối tiếp nhau; một chương trình quy hoạch động riêng cho bài toán cái túi cỡ lớn; và bộ dữ liệu được chuẩn bị sẵn cho bài toán đóng thùng cỡ lớn.**
- **Checker xác minh lời giải của tất cả thuật toán, gồm tham lam, DSATUR, quy hoạch động và quay lui. Với thuật toán heuristic, checker yêu cầu lời giải hợp lệ. Khi có đáp án tham chiếu, kết quả không được tốt hơn đáp án đó; với thuật toán chính xác đã hoàn tất, kết quả phải bằng đáp án tối ưu.**
- **Checker tự xác minh các ràng buộc từ lời giải: không xếp các tàu xung đột chung hợp đồng hoặc đội, đáp ứng đủ kỹ năng, không vượt quỹ giờ và không vượt tải trọng. Chi phí, giá trị hoặc số tàu cũng được tính lại thay vì sử dụng số liệu do thuật toán báo.**
- Sáu testcase không có đáp án tối ưu tham chiếu: m1-L01, m1-L04, m2-L01, m2-L03, m4-L03 và m4-L04. Với các ca này, checker chỉ xác minh tính hợp lệ và tính lại chi phí/số thùng; không thể từ kết quả đó kết luận lời giải tối ưu. Một testcase có đáp án tham chiếu nhưng quay lui chạm giới hạn cũng có thể đạt kiểm tra hợp lệ mà chưa đạt tối ưu.

7.5 Cách chạy

```
python tools/gen_testcases.py   # sinh lại bộ testcase theo seed
python tools/run_testcases.py  # chạy checker, cập nhật RESULTS.md
npm test                       # chạy pytest (102 bài theo LOG.md)
python tests/e2e_modules.py http://127.0.0.1:5173  # E2E sau khi chạy dev server
```

Mỗi testcase có thể dán trực tiếp phần dữ liệu vào giao diện tương ứng để xem thuật toán chạy từng bước.

CHƯƠNG 8. ĐÁNH GIÁ

8.1 Tính đúng đắn: kết quả chạy testcase

Bộ testcase hiện có 88/88 ca đạt theo checker trong testcases/RESULTS.md; 102 bài pytest đạt theo LOG.md. Frontend build thành công và kịch bản E2E Playwright đã chạy qua bốn module. "Đạt" xác nhận tính hợp lệ và các điều kiện đối chiếu áp dụng cho từng ca; không đồng nghĩa mọi thuật toán đều tìm được tối ưu.

Module	Thông thường	Nhỏ	Biên	Đặc biệt	Lớn	Đạt / Tổng
1 Thuê bến	5/5	4/4	9/9	6/6	5/5	29/29
2 Phân công đội	4/4	3/3	5/5	5/5	4/4	21/21
3 Quỹ giờ công	4/4	3/3	5/5	4/4	3/3	19/19
4 Xếp container	3/3	3/3	5/5	4/4	4/4	19/19

Bảng 8.1. Kết quả theo nhóm và module

8.2 Chất lượng của các thuật toán

Testcase	Đáp án tham chiếu	Kết quả thuật toán	Nhận xét
m1-T02 (8 tàu)	2900	Tham lam 3400; tham lam chi phí 3340; DSATUR 3000; quay lui 2900	Bốn thuật toán cho kết quả khác nhau; tất cả hoàn tất.
m2-D04 (2 tàu × 2 đội)	1000	Tham lam không gán hết; khó trước 1000; quay lui 1000	Tham lam theo giờ đến bị kẹt dù bài toán có nghiệm.
m3-T02 (3 yêu cầu, W=50)	220	Tham lam 160; QHĐ 220; quay lui 220	Minh họa tham lam không đảm bảo tối ưu.
m4-D01 (8 lô, C=10)	4 tàu	First-Fit 5; FFD 4; BFD 4; quay lui 4	First-Fit dùng nhiều hơn một tàu.

Bảng 8.2. Một số ca tiêu biểu so sánh thuật toán

- Một số testcase được chọn để cho thấy sự khác nhau giữa các thuật toán. Ở m3-T02, tham lam đạt 160, còn quy hoạch động và quay lui đạt 220. Ở m4-D01, First-Fit dùng 5 thùng; FFD, BFD và quay lui dùng 4. Chỉ kết luận lời giải tối ưu khi có đáp án tham chiếu và thuật toán đã chạy xong.
- Ở module 1, testcase m1-T02 cho bốn kết quả khác nhau: 3400, 3340, 3000 và 2900. Ở module 2, m2-D04 cho thấy tham lam theo giờ đến không gán được hết việc, còn heuristic khó trước và quay lui đạt chi phí 1000. Ở module 4, m4-D02 có tối ưu 3 tàu; FF/FFD/BFD dùng 4, còn quay lui tìm được 3.

- Các thuật toán tham lam chạy nhanh nhưng có thể kém hơn đáp án tham chiếu ở các ca được thiết kế để bộc lộ khác biệt. Quay lui tìm được tối ưu ở các ca có tham chiếu khi hoàn tất. Sáu ca không có đáp án tham chiếu được nêu ở mục 7.4; trong số đó, năm ca không thể kết luận tối ưu chỉ từ checker.

8.3 Thời gian thực hiện

Số liệu được lấy từ cột ms trong RESULTS.md. Với mỗi testcase, báo cáo dùng thời gian chạy cao nhất trong các thuật toán, theo kết quả ghi trên môi trường kiểm thử.

- Có 74/88 testcase có thời gian lớn nhất dưới 10 ms. Trong nhóm ngoài dữ liệu lớn, thời gian cao nhất là 12,55 ms ở m3-B05 (20 yêu cầu, bảng quy hoạch động sát giới hạn).
- Trong nhóm dữ liệu lớn của module 3, thời gian lớn nhất là 14,23 ms ở m3-L03 (25 yêu cầu, W=150); quy hoạch động và quay lui đều đạt giá trị tham chiếu 1519.
- Một số ca lớn hoàn tất dưới giới hạn: m1-L02 (100 tàu) 70,21 ms; m1-L03 (100 tàu) 253,38 ms; m1-L05 (200 tàu nối tiếp) 134,69 ms; m4-L01 (18 lô) 46,62 ms.
- Sáu ca quay lui chạm giới hạn 1,5 giây: m1-L01, m1-L04, m2-L01, m2-L03, m4-L02 và m4-L04. Mỗi lần tìm kiếm còn bị giới hạn ở 400.000 nút. Vì vậy, thời điểm dừng có thể thay đổi theo máy; cần xem các kết quả này theo nhãn “chưa chắc tối ưu”.

8.4 Giao diện và khả năng minh hoạ từng bước

- **Giao diện hỗ trợ chuyển tới hoặc lui từng bước, chạy tự động và kéo thanh trượt. Mã giả làm sáng dòng tương ứng; bảng ứng viên nêu lý do một lựa chọn bị loại, chẳng hạn trùng giờ, thiếu kỹ năng hoặc không đủ tải. Mỗi bước có thông điệp giải thích, còn quay lui hiển thị thêm phương án tốt nhất hiện có và cận.**
- Module 1 còn có biểu đồ Gantt theo hợp đồng, đồ thị xung đột và bảng so sánh bốn thuật toán kèm độ phức tạp.
- **Người dùng có thể nhập dữ liệu trực tiếp, chọn mẫu được tạo từ testcase hoặc nhập và xuất JSON. Module 2 có thêm trường nhập các cặp tàu bị cấm dùng chung đội. Kết quả được gắn nhãn “TỐI ƯU”, “CHƯA CHẮC TỐI ƯU” hoặc “VÔ NGHIỆM”. Bảng thuật ngữ giải thích các cụm như “phương án tốt nhất hiện có” và “bỏ nhánh”.**
- **Các bước được lưu sẵn sau khi tính toán, vì vậy người dùng có thể chuyển qua lại tức thì mà không cần gọi lại máy chủ.**

8.5 Hạn chế

- Quay lui chạm giới hạn ở sáu testcase lớn được liệt kê tại mục 8.3. Trong đó, m4-L02 có đáp án tham chiếu 15 tàu nhưng quay lui dừng ở 16; năm ca còn lại trong danh sách không có đáp án tham chiếu hoặc không thể kết luận tối ưu từ checker.
- Cận cắt nhánh của module 1 chưa tính phần r × thời gian của các tàu chưa được xếp (mục 4.2.3), nên có thể chưa loại được nhiều nhánh không cần thiết.

- **Bước phát hiện vô nghiệm ở module 2 mới kiểm tra điều kiện cần và chưa xét các cặp tàu bị cấm bổ sung. Vì vậy, một số trường hợp vô nghiệm vẫn cần thuật toán quay lui tìm kiếm để kết luận.**
- Giới hạn dữ liệu đầu vào hiện tại: module 1 tối đa 200 tàu; module 2 tối đa 60 tàu và 12 đội; module 3 tối đa 25 yêu cầu, $W \le 200$ và $n \times (W + 1) \le 4.000$ ô; module 4 tối đa 60 lô. Mỗi thuật toán trả tối đa 3.000 bước, giảm xuống khi dữ liệu lớn. Quay lui bị giới hạn ở 400.000 nút và 1,5 giây.
- Sáu testcase không có đáp án tham chiếu chỉ được xác minh tính hợp lệ và tính lại kết quả; chúng không chứng minh tối ưu.
- Thời gian trong RESULTS.md được đo trên một máy, không phải trung bình của nhiều lần chạy. Các ca chạm giới hạn thường mất khoảng 1,5 giây; số đo có thể khác giữa các máy.
- Giao diện hiện chưa có trang Tổng quan hoặc nút chuyển danh sách tàu từ module 1 sang module 2.

8.6 Hướng phát triển

- Bổ sung vào cận quay lui của module 1 phần $r \times$ thời gian của các tàu chưa xếp để tăng khả năng cắt nhánh và giảm số testcase chạm giới hạn.
- Mở rộng bước kiểm tra vô nghiệm của module 2 để xét cả các cặp tàu bị cấm dùng chung đội.
- Khôi phục trang Tổng quan và bổ sung chức năng chuyển danh sách tàu từ module 1 sang module 2.
- Thêm các ca kiểm tra cố ý làm sai lời giải để xác nhận checker độc lập phát hiện và từ chối kết quả không hợp lệ.
- Bổ sung thuật toán so sánh, chẳng hạn nhánh cận với thứ tự duyệt tốt hơn, đồng thời đo thời gian qua nhiều lần chạy.

8.7 Tổng hợp mối liên hệ và kết luận

Bài toán	Mô hình	Cấu trúc dữ liệu	Thuật toán	Demo	Testcase / đánh giá
Thuê bến	Tô màu đồ thị khoảng, chi phí F + r·khoảng phủ	Danh sách kề, mảng màu, lo/hi/cnt của hợp đồng	Tham lam, hợp đồng rẻ nhất, DSATUR, quay lui	Phát từng bước, biểu đồ Gantt, đồ thị xung đột	27 ca, đạt 27/27
Phân công đội	Gán việc có danh sách cho phép	Danh sách đội đủ kỹ năng, tập xung đột, cận suffix	Tham lam (2 kiểu), quay lui, kiểm tra Hall	Bảng ứng viên có lý do loại	18 ca, đạt 18/18
Quỹ giờ công	Cái túi 0/1	Bảng dp, mảng take	Tham lam, QHĐ, quay lui + cận phân số	Bảng dp từng ô, truy vết	17 ca, đạt 17/17
Xếp container	Đóng thùng	Mảng gán bin, mảng tải loads	FF, FFD, BFD, quay lui	Thùng xếp theo từng lô	17 ca, đạt 17/17

Bảng 8.3. Mối liên hệ bài toán – mô hình – cấu trúc dữ liệu – thuật toán – demo – testcase

Đồ án hiện có demo cho bốn bài toán điều phối cảng, các thuật toán có thể so sánh và phát lại từng bước. Theo hồ sơ kiểm thử hiện tại, 88 testcase đạt checker, 102 pytest đạt, frontend build thành công và E2E Playwright đạt. Các ca lớn chạm giới hạn được ghi rõ; kết quả của chúng không được gọi là tối ưu nếu chưa có chứng minh hoặc đáp án tham chiếu.

Công nghệ của project hiện tại: Python với FastAPI, Uvicorn, pytest và httpx; frontend dùng React 18.3.1 và Vite 5.4; E2E dùng Playwright. Cấu hình nằm trong backend/requirements.txt, frontend/package.json và tests/e2e_modules.py.

PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ CẤU TRÚC MÃ NGUỒN

A.1 Chạy trên Codespaces hoặc trên máy

Mã nguồn: github.com/SerenaVo-code/portopt. Tại thư mục gốc của project, chạy các lệnh sau trong Terminal:

```
npm run setup    # cài thư viện (một lần)
npm run dev      # chạy backend :8000 + giao diện :5173 (Ctrl+C để dừng)
```

Mở tab Ports, bấm biểu tượng địa cầu ở cổng 5173. Trên máy cá nhân cũng dùng đúng các lệnh này.

A.2 Cấu trúc thư mục

```
(đường dẫn tính từ thư mục gốc)
backend/solver.py       xử lý chung và phát lại bước
backend/berth.py        module 1: thuê bến
backend/assign.py       module 2: phân công đội
backend/knapsack.py     module 3: quỹ giờ công
backend/binpack.py      module 4: xếp container
backend/server.py       API FastAPI
frontend/src/           giao diện React/Vite
tests/                  pytest và E2E Playwright
testcases/m*/           88 testcase theo bốn module
testcases/RESULTS.md    kết quả checker
tools/                  sinh/chạy testcase và checker
```

PHỤ LỤC B. PROMPT TIÊU BIỂU TRONG QUÁ TRÌNH PHÁT TRIỂN

#	Prompt	AI phản hồi	Quyết định sau khi AI phản hồi
1	Mình đã thiết kế giao diện trong các hình đính kèm. Hãy xem kỹ bố cục, màu sắc, kiểu chữ, khoảng cách và cách sắp xếp các thành phần, rồi chỉnh giao diện trong project để bám sát thiết kế đó	AI được yêu cầu chỉnh giao diện theo bố cục, màu sắc, kiểu chữ, khoảng cách và cách sắp xếp thể hiện trong hình.	
2	Hãy đọc đề bài, tài liệu mô tả và code hiện có của bốn module. Rà soát từng thuật toán xem đã đúng với mô hình bài toán, ràng buộc và mục tiêu tối ưu chưa; kiểm tra cả tính đúng đắn, độ phức tạp và các trường hợp biên. Trước tiên, hãy nêu những điểm chưa đúng hoặc có thể cải thiện, kèm file và lý do	Mình đã đối chiếu các thuật toán với mô tả và kết quả kiểm thử hiện có. Có một số điểm có thể cải thiện; mình **chưa sửa code**: 1. **Module thuê bến:** cận cắt nhánh hiện chưa tính phần chi phí theo thời gian của các tàu chưa xếp. Bổ sung thành phần này có thể giúp loại thêm nhánh tìm kiếm. 2. **Module phân công đội:** bước kiểm tra Hall chỉ là điều kiện cần và chưa bao quát các cặp tàu bị cấm. Có thể mở rộng bước kiểm tra sớm để phát hiện thêm trường hợp vô nghiệm. 3. **Module quỹ giờ công:** giới hạn bảng quy hoạch động đang phù hợp cho demo, nhưng cần kiểm tra rõ các ca chạm giới hạn $n \times (W + 1) \leq 4.000$ và cách giao diện thông báo khi dữ liệu vượt giới hạn. 4. **Module xếp container:** một số ca quay lui lớn chạm giới hạn thời gian; có thể cải thiện thứ tự thử lô hoặc cận dưới để tìm lời giải tốt hơn sớm hơn	Đồng ý, thực hiện 4 ý đó.

#	Prompt	AI phản hồi	Quyết định sau khi AI phản hồi
	Mình đã tự phác thảo các testcase trong thư mục testcases. Hãy xem từng ca, giữ lại những ca hợp lý và hoàn thiện dữ liệu hoặc mô tả còn thiếu. Sau đó đề xuất thêm testcase để bao phủ các trường hợp thông thường, biên, đặc biệt và dữ liệu lớn theo ràng buộc của từng module	Mình đã chia các ca bạn phác thảo theo bốn module và thấy có thể rà thêm các nhóm sau: • **Thuê bến:** tàu nối tiếp nhau không chồng giờ; tàu qua đêm; chi phí cố định hoặc đơn giá bằng 0; xung đột do cặp bị cấm. • **Phân công đội:** đội thiếu kỹ năng; các tàu trùng giờ cần cùng một đội; cặp tàu bị cấm; trường hợp không có đội phù hợp. • **Quỹ giờ công:** quỹ bằng 0; chọn vừa khít quỹ; có yêu cầu giá trị 0; ca mà tham lam cho kết quả kém hơn quy hoạch động. • **Xếp container:** lô bằng đúng tải trọng; nhiều lô cùng khối lượng; ca tham lam dùng nhiều tàu hơn đáp án tốt nhất; dữ liệu lớn gần giới hạn.	Ok. Bổ sung mấy testcase còn thiếu

Bảng B.1. Một số prompt và phản hồi trong quá trình phát triển